

TransitFlow AI TOOL USAGE 說明

1.密碼儲存問題以及多模態整合問題

CONTEXT:

 維度四:資訊安全 (Security)CONTEXT

這是資安紅線, 犯了通常直接零分或不及格:

密碼儲存: 絕對不能明文儲存密碼! 資料庫欄位必須預留 60 個字元(為 bcrypt 或 argon2id 雜湊演算法做準備)。

防範 SQL 注入攻擊 (SQL Injection): 在 queries.py 中撰寫查詢時, 嚴禁使用 Python 的字串拼接 (f-string 或 +)。必須嚴格使用參數化查詢 (SQL 用 %s, Neo4j 用 \$param)。

環境變數: 連線字串必須放在 .env 檔案中讀取, 不能直接 Hard-code 寫死在 Python 程式碼裡。

 維度五:多模態資料庫整合 (Polyglot Persistence)

這項作業最大的亮點, 就是你是否能讓 AI Agent 根據問題「選對工具」:

PostgreSQL: 負責處理高度結構化的交易、使用者與時刻表。

Neo4j (圖資料庫): 是否能正確載入車站拓樸圖, 讓 AI 能計算最短路徑與轉乘站 (避開 SQL 複雜的遞迴 CTE)。

pgvector: 政策文件是否有正確被轉為向量存入, 並能透過餘弦相似度 (Cosine Similarity) 被成功檢索。

PROMPT:

請你在確保程式能正確運行的前提下幫我對程式碼進行修改, 並且嚴格符合以上條件。

OUTCOME:

這兩個維度是決定你專案成敗的關鍵:「資訊安全」是不可踩的紅線(踩了直接不及格), 而「多模態資料庫整合」則是拿高分的核心亮點。

要確保這兩個維度的正確性, 你可以透過「程式碼自我審查 (Code Review)」與「Agent 實機測試」來進行驗證。以下我為你整理了具體的檢查清單與驗證方法:

維度四:資訊安全 (Security) 檢查清單

這個維度主要靠檢查你的 Python 與 SQL 程式碼來確保。請打開你的編輯器, 逐一核對以下三點:

1. 密碼儲存 (Password Storage)

- 去哪裡檢查: `databases/relational/schema.sql`
- 如何確認正確: 找到你的 `users` 表格, 確認密碼欄位的長度與命名。
 -  錯誤示範: `password VARCHAR(20)` (長度太短, 且名稱暗示明文)
 -  正確示範: `password_hash VARCHAR(60)` (明確標示為 hash, 且長度足夠容納 bcrypt 產生的 60 碼字串)

2. 防範 SQL/Cypher 注入攻擊 (Injection Prevention)

- 去哪裡檢查: `databases/relational/queries.py` 與 `databases/graph/queries.py`
- 如何確認正確: 使用編輯器的搜尋功能(Ctrl+F 或 Cmd+F), 在這些檔案中搜尋 `f"` (f-string) 或 `%` (舊式字串格式化)。在 `execute()` 裡面絕對不能出現這兩個東西。
 -  危險的 PostgreSQL (零分): `cur.execute(f"SELECT * FROM users WHERE email = '{email}')`
 -  安全的 PostgreSQL (滿分): `cur.execute("SELECT * FROM users WHERE email = %s", (email,))` (注意後面的 *tuple* 逗號)
 -  危險的 Neo4j (零分): `session.run(f"MATCH (n:Station {{name: '{station_name}'}}) RETURN n")`
 -  安全的 Neo4j (滿分): `session.run("MATCH (n:Station {name: $name}) RETURN n", name=station_name)`

3. 環境變數 (Environment Variables)

- 去哪裡檢查: `databases/relational/queries.py` 等需要連線的檔案。
- 如何確認正確: 確認你沒有把資料庫密碼寫死在程式碼裡。
 -  錯誤示範: `conn = psycopg2.connect("dbname=transit user=postgres password=mysecretpassword")`
 -  正確示範: 專案中應該有類似 `from skeleton.config import PG_DSN` 的設計, 透過設定檔去讀取 `.env` 中的 `DATABASE_URL`。



維度五: 多模態整合 (Polyglot Persistence) 檢查清單

這個維度不僅看資料庫有沒有建好, 更要看 **AI Agent** 能不能正確地「路由(Routing)」問題給對的資料庫。

1. 如何驗證 PostgreSQL (結構化交易)

- 測試問題: 在 UI 聊天介面輸入:「我的帳號 (*user_id: RU01*) 有哪些國鐵的訂票紀錄?」
- 如何確認正確: 打開 UI 右側的 **Debug Panel** (除錯面板)。你應該要看到 AI 選擇呼叫了類似 `get_user_bookings` 的工具, 並且撈出的是 JSON 格式的訂單資料。這證明 AI 知道「交易與個人資料歸 **PostgreSQL** 管」。

2. 如何驗證 Neo4j (圖資料庫與路徑計算)

- 測試問題: 在 UI 聊天介面輸入:「請問從 *Central Station (NR01)* 轉乘到 *Sunnyvale (MS18)* 最快的路徑是什麼? 需要多久?」
- 如何確認正確: 1. 看 **Debug Panel**: AI 必須呼叫圖資料庫的工具(例如 `find_shortest_path`), 而不是去撈關聯式資料庫的班表。2. 看 **Cypher** 語法: 確認你的 Neo4j 查詢有利用到圖論演算法, 例如使用 Dijkstra 演算法(`apoc.algo.dijkstra`)或是基於關係權重(`travel_time_min`)的最短路徑查詢, 而不是用 PostgreSQL 寫出極度複雜又慢的 Recursive CTE。

3. 如何驗證 pgvector (政策向量檢索)

- 測試問題: 在 UI 聊天介面輸入:「如果我的國鐵特快車因為營運商的問題延誤了 65 分鐘, 我可以獲得多少退費?」
- 如何確認正確:
 1. 看 **Debug Panel**: AI 必須呼叫 `query_policy_vector_search` (或類似名稱的 RAG 工具)。
 2. 看 **SQL** 語法: 確認你的 `schema.sql` 裡有 `CREATE EXTENSION IF NOT EXISTS vector;`, 並且在查詢政策時, 有使用餘弦相似度運算子 `<=>`。
 3. 看精準度: AI 回答必須精準命中 `refund_policy.json` 中關於「延誤 60-119 分鐘」的補償條款, 而不是亂編答案。

💡 終極心法: **Agent** 的大腦是「工具描述 (Tool Descriptions)」

如果你的 Agent 老是「選錯工具」(例如問路徑, 它卻跑去查 PostgreSQL), 請去檢查 `skeleton/agent.py` (或是註冊工具的地方) 的「工具描述」。

LLM 是透過你寫的英文描述來決定要用哪個工具的。你可以把描述寫得非常明確 (Prompt Engineering):

- **PostgreSQL** 工具描述: "Use this tool ONLY to find user accounts, schedules, ticket prices, and booking history."
- **Neo4j** 工具描述: "Use this tool ONLY when the user asks for the fastest route, shortest path, or how to transfer between stations."
- **Vector** 工具描述: "Use this tool ONLY when the user asks about rules, refund policies, luggage limits, or passenger conduct."

只要你的程式碼守住資安底線, 且 Agent 能夠依照上述邏輯完美切換三把武器, 這個專案就具備了極高的技術水準!